

ZeroMint AIOS Engineering Evaluation 1.0

Artifact Distribution, Integrity Evidence, Installation Boundaries, and Supply-Chain Gaps

Shafaet Brady Hussain

Independent researcher and developer, United Kingdom

18 August 2026

Abstract

I evaluate the publicly observable engineering evidence for ZeroMint OS v1.0 and its OpenZero AIOS installation lane. The evaluated release is a 5,945,425,920-byte ISO distributed through a direct HTTPS endpoint, a BitTorrent v1 metainfo file, and three GitHub release parts. Live GitHub metadata exposes SHA-256 digests for all three parts and the torrent; the part sizes sum exactly to the documented ISO size. The direct endpoint returned HTTP 200 with the same content length. The local torrent is valid bencoded metadata naming the ISO, declaring the same length, a 2 MiB piece size, and 2,835 SHA-1 piece hashes. A related Bash wrapper passed syntax validation and its help path executed without privileged changes. These checks establish coherent distribution metadata and guarded installer entry conditions, not operating-system correctness. I did not download or mount the ISO, reassemble all parts, boot a virtual machine, inspect packages, reproduce the image, or validate the claimed internal system. The public release lacks a cryptographic signature, SBOM, build recipe, provenance attestation, and rollback-resistant update metadata. I therefore classify ZeroMint AIOS as a documented distribution artifact with useful integrity checks but insufficient evidence for claims of a reproducible, hardened, or independently validated operating system.

1. Evaluation question

The central question is narrow: what can the public artifacts prove about ZeroMint AIOS engineering, and what remains unverified? I treat “AIOS” as product terminology for an OpenZero-focused operating-system-style distribution route. I do not treat it as a new operating-system kernel, a formally defined computing category, or evidence of autonomous intelligence.

The evaluation covers release packaging, artifact identity, transport alternatives, installer guardrails, documentation, and missing supply-chain controls. It excludes confidential infrastructure, paid code, private prompts, credentials, customer data, and server internals.

2. Evidence snapshot

The source documentation records an ISO SHA-256 of 52f2d62f7f286484b28f7c5128b398c1ddb87ca354efa997965f5eef98263668 and torrent SHA-256 of 04c02071a827b9af0a5b8883b2627edfb26b6f98e6907b5bc737fabdf66185e7. The local torrent hash matched this value. GitHub's release API independently reported the same torrent asset digest and the following three part digests.

Release asset	Bytes	GitHub SHA-256 digest
ZeroMint_OS_v1.0.iso.part001	1,992,294,400	95cbb9afd3841b6c2c0dcfca107fe05b389c5345dbbfc757d58e153482f51308
ZeroMint_OS_v1.0.iso.part002	1,992,294,400	d3fcad548bec2ea3c844e2e73cfba7ac83a4502778bce5b204d7c14f963567f3
ZeroMint_OS_v1.0.iso.part003	1,960,837,120	c76d919b7f3b186c3310bfd50d1f9ad194ed5a541723ca46ab4dc25f844e6a3c
Sum	5,945,425,920	Equals documented ISO length

The GitHub release was published on 4 July 2026. At evaluation time, the direct HTTPS ISO endpoint returned status 200, media type application/octet-stream, and content length 5,945,425,920. A successful HEAD response establishes current discoverability and size metadata, not byte identity.

3. Distribution architecture

3.1 Split GitHub release

The ISO is divided into three assets below GitHub's per-asset limit. The documentation gives reassembly commands for Unix-like systems and Windows, followed by a whole-image SHA-256 check. This is operationally useful because part digests localize a corrupted download while the whole-image digest verifies the reassembled result.

3.2 Direct HTTPS route

The direct server provides a simpler single-file path. HTTPS protects transport to the authenticated domain, but the same origin also publishes the checksum text. If that origin is compromised, both artifact and checksum could be replaced. An external signature or independently anchored transparency record would improve this boundary.

3.3 BitTorrent route

The local 56,944-byte metainfo file parsed completely as a BEP 3 dictionary. It declares ZeroMint_OS_v1.0.iso, length 5,945,425,920, piece length 2,097,152, 2,835 SHA-1 piece hashes, and the udp://tracker.opentrackr.org:1337/announce tracker. This supports piece-level transfer integrity under BitTorrent v1. It does not replace the separately published SHA-256 whole-file check or authenticate the publisher.

4. Installer boundary evaluation

A public FreeWebPanel wrapper provides an adjacent OpenZero AIOS and ZeroMint Linux web-server lane. Its SHA-256 in the evaluated local snapshot is a206588b49812f0f0c7073af18a563a3bbb9f28311119c120854ad6ad69113bc. Bash syntax validation passed, and --help executed without making system changes.

The script uses `set -euo pipefail`, requires explicit hostname and email arguments, checks for root privileges, restricts supported operating-system identifiers to Linux Mint or Ubuntu, validates the selected profile, creates a temporary directory with `mktemp -d`, and removes it on exit. It offers a `--preflight-only` path and downloads the upstream installer over HTTPS before executing it.

These are useful fail-fast controls, but significant risks remain:

- the upstream installer and bundle URLs are mutable network locations;
- no pinned digest or signature is checked before execution;
- piping a remote script into privileged Bash concentrates trust in DNS, TLS, hosting, and the current remote bytes;
- syntax validation does not establish correct behavior on a real host;
- this evaluation did not run the privileged preflight or installation path.

5. Comparison with established practice

SHA-256 is standardized in NIST FIPS 180-4 and is suitable for detecting byte changes when the expected digest is trusted. BEP 3 defines BitTorrent v1 metainfo, fixed-size pieces, and the information hash. ZeroMint uses both mechanisms coherently for transport and whole-file verification.

However, a checksum is not a publisher signature. Modern update frameworks such as The Update Framework separate signed root, targets, snapshot, and timestamp roles to address key compromise, rollback, freeze, and mix-and-match attacks. NIST's Secure Software Development Framework also emphasizes retained release integrity, provenance, protected build environments, and vulnerability response. ZeroMint's public snapshot does not yet expose equivalent metadata.

6. Engineering scorecard

Control	Observed result	Evaluation
Whole-image SHA-256 documented	Present	Useful if digest source is trusted
GitHub part digests	Present and live	Strong part-level identity evidence
Part-size arithmetic	Exact match	Packaging is internally coherent
Torrent metadata	Valid BEP 3 structure	Alternative transfer route is coherent
Direct endpoint	HTTP 200 and expected length	Availability evidence only
Reassembly instructions	Windows and Unix paths	Practical operator guidance
Installer syntax and help	Passed	Entry-path evidence only
Detached publisher signature	Not found	Major authenticity gap
SBOM and package inventory	Not found	Composition cannot be audited publicly
Reproducible image build	Not found	ISO cannot be independently rebuilt
Boot/install test evidence	Not performed here	Runtime behavior unverified
Signed update metadata	Not found	Rollback/freeze defenses unverified

7. Release gates for a stronger 1.1

1. publish a deterministic image-build recipe with pinned upstream repositories and package versions;

2. generate CycloneDX or SPDX SBOMs for the image and bundled OpenZero components;
3. sign release manifests with a documented offline or hardware-backed publisher key;
4. add provenance attestation linking source revision, builder image, commands, and output hash;
5. run automated virtual-machine boot, install, reboot, network, local-panel, and shutdown tests;
6. scan packages and configuration against a declared vulnerability baseline;
7. adopt signed, expiry-aware, rollback-resistant update metadata; and
8. publish third-party licence notices and redistribution conclusions for every included component.

8. Threats to validity and limitations

- The ISO was not downloaded, mounted, or executed in this evaluation.
- GitHub asset digests verify hosted assets, but the whole ISO hash remains documentation-derived in this run.
- HEAD metadata can be correct while response bytes are wrong or change during transfer.
- The torrent's piece hashes use SHA-1 as defined for BitTorrent v1; this is transport metadata, not publisher authentication.
- No seed availability or full torrent transfer was measured.
- The installer wrapper was not run as root, so package, firewall, DNS, SSL, and backup behavior remain untested.
- Absence of a public artifact in the inspected corpus is not proof that no private process exists.
- Product branding and web-page claims are not engineering evidence unless tied to executable tests.

9. Falsifiable claims

1. The three GitHub part sizes sum to 5,945,425,920 bytes.
2. The live GitHub release API returns the three recorded part digests and torrent digest.
3. The evaluated torrent parses completely and declares the documented ISO name and length.
4. The torrent contains 2,835 piece hashes at a 2 MiB piece length, consistent with the declared file size.
5. The installer passes `bash -n` and its help path exits without privileged changes.
6. The evaluated public package contains no detached signature, SBOM, reproducible-build recipe, or signed update metadata.

Claims 1-5 are directly testable with the public verifier and named artifacts. Claim 6 is bounded to the inspected release snapshot and should be retested when the project changes.

10. AI-use disclosure

I used OpenAI Codex to inventory public and local release evidence, query the GitHub release API, parse torrent metadata, run non-privileged syntax checks, compare the design with primary specifications, draft the manuscript, and build the release package. I remain responsible for its claims and corrections. AI

assistance is not peer review or independent validation.

11. Licensing and availability

The manuscript and public verification metadata are released under CC BY 4.0; the verifier is MIT licensed. The ZeroMint ISO is referenced by hash and public URL but is not redistributed in this research package. This paper does not determine the redistribution rights of the ISO or its components.

References

1. NIST, "Secure Hash Standard (SHS)," FIPS 180-4, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>
2. B. Cohen, "The BitTorrent Protocol Specification," BEP 3. https://www.bittorrent.org/beps/bep_0003.html
3. The Update Framework, "Specification." <https://theupdateframework.org/spec/>
4. NIST, "Secure Software Development Framework, SP 800-218." <https://csrc.nist.gov/projects/ssdf>
5. ResearchForumOnline, "ZeroMint OS v1.0 GitHub Release." <https://github.com/ResearchForumOnline/OpenZero/releases/tag/zeromint-os-v1.0>

Appendix A. Evidence ledger

Evidence	SHA-256 or value	Boundary
ZeroMint documentation snapshot	5e18dbfa098d09c53fd7f9905d85bb6c badaf12bf028e81ba8a80ef00b47ce11	Text and declared hashes
Local torrent	04c02071a827b9af0a5b8883b2627edf b26b6f98e6907b5bc737fabdf66185e7	Metadata only
Installer wrapper	a206588b49812f0f0c7073af18a563a3 bbb9f28311119c120854ad6ad69113bc	Non-privileged checks only
GitHub release	Published 2026-07-04	Live asset metadata checked 2026-08-18
Direct ISO endpoint	200; 5,945,425,920 bytes	HEAD response, not content hash
Boot/install behavior	Not tested	No operational claim
Peer review	None	Public working paper